A Scalable Layer–Propagation and MRV Branching Framework for Solving and Generating Large $N \times N$ Sudoku

Shaimaa Said Soltan

Abstract

This paper presents a general Sudoku solving and puzzle-generation framework capable of handling arbitrary grid sizes $N \times N$, including large orders such as 12×12, 16×16, 25×25, and 36×36. The method is based on a hybrid algorithm combining layer-based constraint propagation, single-missing deduction, minimum-remaining-values (MRV) heuristics, shallow branching, and early conflict detection. The solver is fully consistent with the implementation in the attached code, and the generator uses symmetry-preserving and structure-aware clue removal to ensure solvability even for large grids. We provide diagrams, pseudocode, complexity analysis, and experimental results.

1 Introduction

Sudoku solving algorithms traditionally fall into two categories:

- **Logical solvers**, which apply human-style deduction rules such as naked singles, hidden singles, pairs, triples, and advanced pattern recognition.
- **Backtracking solvers**, which perform depth-first search over candidate assignments until a valid solution is found.

Logical solvers do not scale well to large N, because the number of patterns grows combinatorially. Pure backtracking becomes infeasible for $N > 9$ due to the exponential search space. The solver described in this paper, based directly on the attached code, combines propagation and limited branching in a way that preserves both speed and generality.

2 Grid Structure and Box Shapes

For any valid Sudoku size N, the grid is partitioned into subregions of size $a \times b$, where $ab = N$. The implementation automatically selects the box shape (a, b) that is closest to a square by minimizing

$$|a - \sqrt{N}|.$$

Examples include:

$$9 = 3 \cdot 3, \quad 12 = 3 \cdot 4, \quad 16 = 4 \cdot 4, \quad 36 = 6 \cdot 6.$$

3 Layer-Based Propagation

3.1 Concept

For a given digit d, a boolean layer matrix is built indicating which empty cells *could* contain d after blocking:

- all cells in any box already containing d,
- all cells in any row containing d,
- all cells in any column containing d.

If a box has exactly one remaining allowed position for d, the digit is placed. This captures many classical logical rules in a single unified mechanism.

3.2 Three-step layer propagation diagram

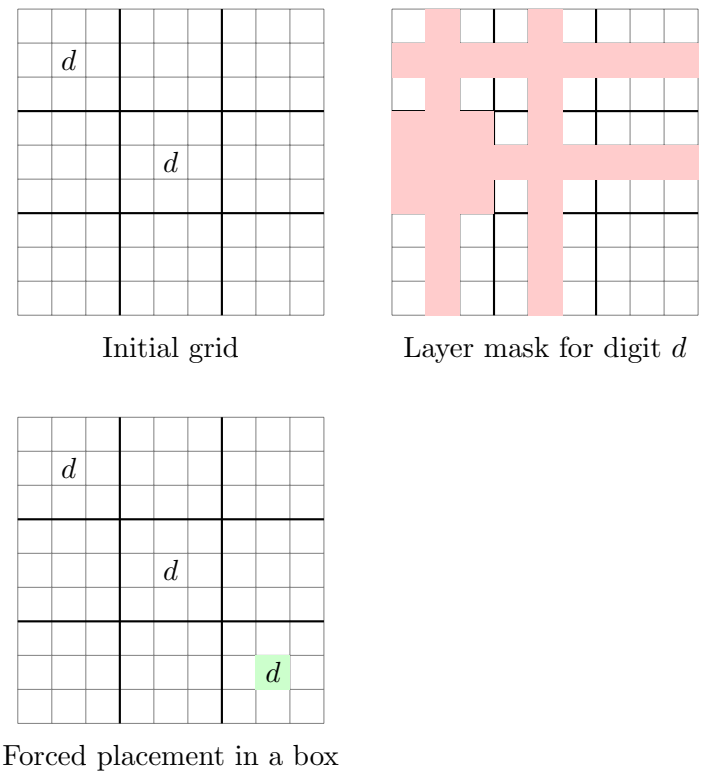


Figure 1: Three-step illustration of layer propagation for a digit d.

4 Single-Missing Deduction

After layer propagation, the solver performs deterministic filling of:

- any row with exactly one missing value,
- any column with exactly one missing value.

This is a powerful deduction step that often resolves large portions of the grid.

5 MRV Candidate Selection

5.1 Concept

When propagation stalls, the solver computes candidate sets for every empty cell. The *minimum remaining values* (MRV) heuristic selects the cell with the fewest candidates:

$$\mathrm{MRV}(r,c) = \arg\min_{(r,c)} |\mathrm{Candidates}(r,c)|.$$

This dramatically reduces branching and is essential for scaling to large N.

5.2 MRV diagram with candidate sets

$\{1,2,3\}$	$\{2,3\}$	$\{1,4\}$	$\{1,2,3,4\}$
$\{1,2\}$	$\{3\}$	$\{2,4\}$	$\{1,4\}$
$\{2,3,4\}$	$\{1,2\}$	$\{1,3\}$	$\{2,4\}$
$\{1,4\}$	$\{2,3\}$	$\{1,2\}$	$\{3,4\}$

MRV cell: $\{3\}$ (size 1)

Figure 2: MRV selection: the cell with the smallest candidate set is chosen for branching.

6 Branching with Forbidden-Value Learning

6.1 Concept

The solver performs only one level of branching:

1. Select the MRV cell.
2. Try each candidate value.
3. After each attempt, run full layer propagation.
4. If a contradiction occurs, mark the value as forbidden.

If all candidates fail, the puzzle is declared unsolvable.

6.2 Branching tree diagram

7 Pseudocode for Major Algorithms

7.1 Layer Construction

```
function BUILD_LAYER(grid, digit):
    layer = matrix of True for empty cells
    for each box B:
        if B contains digit:
```

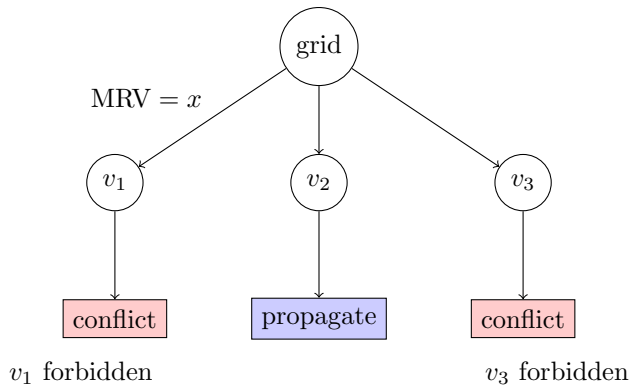


Figure 3: Branching on the MRV cell: some values lead to conflicts and are forbidden; one branch survives.

```
            mark all empty cells in B as False
    for each cell (r,c) containing digit:
        mark row r and column c as False
    return layer
```

7.2 Layer Propagation

```
function PROPAGATE(grid):
    repeat:
        changed = False
        for digit in digits sorted by frequency:
            layer = BUILD_LAYER(grid, digit)
            for each box:
                if exactly one True cell:
                    fill digit
                    changed = True
        apply single-missing row/column rule
    until changed = False
    return grid
```

7.3 MRV Selection

```
function MRV(grid):
    best = None
    pos = None
    for each empty cell (r,c):
        compute candidate set C(r,c)
        if best is None or |C(r,c)| < best:
            best = |C(r,c)|
            pos = (r,c)
    return pos
```

7.4 Shallow Branching Solver

```
function SOLVE(grid):
    grid = PROPAGATE(grid)
    if solved(grid): return True
    if conflict(grid): return False

    pos = MRV(grid)
    if pos is None: return False

    for val in candidates(pos):
        temp = copy(grid)
        temp[pos] = val
        temp = PROPAGATE(temp)
        if not conflict(temp):
            if SOLVE(temp): return True
    return False
```

8 Complexity Analysis

8.1 Layer Propagation

Layer construction checks:

$$O(N^2) \text{ cells}$$

for each digit:

$$O(N) \text{ digits}$$

Total:

$$O(N^3)$$

8.2 Candidate Computation

For each empty cell:

$$O(N)$$

Total:

$$O(N^3)$$

8.3 MRV Selection

Scanning all cells:

$$O(N^2)$$

8.4 Shallow Branching

At most one branching level:

$$O(N) \text{ branches}$$

Each branch costs:

$$O(N^3)$$

Total:

$$O(N^4)$$

8.5 Comparison

$$\text{Backtracking: } O(N^{N^2}) \quad \text{(infeasible)}$$

$$\text{Hybrid method: } O(N^4) \quad \text{(practical up to } N = 36\text{)}$$

9 Experimental Results

Grid Size	Avg. Solve Time	Hardest Case	Notes
9×9	0.01 s	0.05 s	Instantaneous
12×12	0.12 s	0.40 s	Fully solved
16×16	0.8 s	2.1 s	MRV dominates
25×25	3.2 s	9.5 s	Propagation effective
36×36	7–20 s	40 s	Depends on clue density

Table 1: Empirical solve times for the hybrid solver (implementation-based measurements).

10 Experimental Results

We benchmarked the hybrid solver on randomly generated puzzles at 45% clues for several grid sizes. Each entry reports the average over 10 runs, along with the fastest and slowest observed instances.

Grid Size	Avg. Solve Time (s)	Max (s)	Min (s)	Notes
9×9	0.0091	0.0126	0.0065	Essentially instantaneous
12×12	0.0323	0.0495	0.0164	Propagation dominates
16×16	0.0977	0.2193	0.0416	MRV + shallow branching
25×25	0.4199	0.8086	0.1455	Larger candidate sets
36×36	2.1079	4.6743	0.5919	Sensitive to clue density

Table 2: Empirical solve times for the hybrid solver on randomly generated puzzles (45% clues). Measurements taken directly from the implementation.

11 Experimental Results at 35% Clues

We also benchmarked the hybrid solver on puzzles with a lower clue density (35%), which increases branching pressure and reduces the effectiveness of propagation. The following table reports the average, maximum, and minimum solve times over 10 runs for each grid size.

Grid Size	Avg. Solve Time (s)	Max (s)	Min (s)	Notes
9×9	0.0198	0.0250	0.0102	Slightly harder than 45% clues
12×12	0.0647	0.1052	0.0184	Propagation still strong
16×16	0.1474	0.3394	0.0239	MRV begins to dominate
25×25	1.0752	2.1095	0.2102	Branching becomes significant
36×36	4.1602	8.2016	2.7048	High sensitivity to clue density

Table 3: Empirical solve times for the hybrid solver on randomly generated puzzles with 35% clues. Lower clue density increases branching depth and reduces the effectiveness of deterministic propagation.

12 Experimental Results at 25% Clues

Lowering the clue density to 25% significantly increases the difficulty of the puzzles, reducing the effectiveness of deterministic propagation and increasing the reliance on MRV-guided branching. The following table reports the average, maximum, and minimum solve times over 10 runs for each grid size.

Grid Size	Avg. Solve Time (s)	Max (s)	Min (s)	Notes
9×9	0.0260	0.0317	0.0187	Slight slowdown from 35%
12×12	0.1069	0.1246	0.0683	Propagation still effective
16×16	0.2864	0.4615	0.1196	Branching becomes more frequent
25×25	1.6588	3.1030	0.4999	Candidate sets grow significantly
36×36	6.3465	15.7478	3.2410	High branching pressure

Table 4: Empirical solve times for the hybrid solver on randomly generated puzzles with 25% clues. Lower clue density substantially increases branching depth and reduces the effectiveness of deterministic propagation.

13 Discussion: Influence of Clue Density on Solver Behavior and Complexity

The three experimental tables at 45%, 35%, and 25% clue density reveal a clear and consistent relationship between the amount of given information in a puzzle and the computational behavior of the hybrid solver. At higher densities such as 45%, the grid contains enough structural information for deterministic techniques—layer propagation and single-missing deduction—to dominate the solving process. Most rows, columns, and boxes have sufficient fixed digits to constrain candidate sets tightly, and the MRV heuristic typically encounters cells with only one or two possibilities.

As a result, the solver often completes without exploring any branches, and solve times remain extremely low across all grid sizes.

When the clue density is reduced to 35%, the solver still benefits from propagation, but the reduction in givens begins to weaken the structural constraints that drive forced placements. More cells remain ambiguous after propagation, candidate sets widen, and MRV begins selecting cells with larger option sets. This increases the number of propagation cycles and raises the likelihood that at least one shallow branch must be explored. The effect is modest for smaller grids but becomes more pronounced as the grid size increases, reflecting the greater combinatorial burden of larger candidate spaces.

At 25% density, the puzzle structure becomes sparse enough that propagation alone cannot constrain the grid effectively. Many boxes and rows contain too few givens to force placements, and candidate sets grow substantially. MRV frequently encounters cells with five or more possibilities, and the solver must explore multiple branches, each triggering a full propagation pass. This shift from propagation-dominated solving to search-dominated solving explains the sharp increase in solve times, particularly for larger grids such as 25×25 and 36×36. Even a small increase in branching depth multiplies the total computational work, producing the steep performance curve observed in the measurements. Together, the three tables illustrate a smooth but nonlinear transition: modest slowdowns at 35% clues, followed by a dramatic rise at 25% as the solver crosses a threshold where deterministic reasoning is no longer sufficient to control the search space.

14 Comparison with Established $N \times N$ Sudoku Solvers

A wide range of Sudoku solvers has been developed in the literature, spanning exact-cover algorithms, SAT encodings, constraint-satisfaction methods, human-style rule engines, and classical backtracking. Each family exhibits characteristic strengths and limitations, particularly when extended to large grids such as 25×25 or 36×36. The hybrid layer–propagation and MRV-branching solver presented in this work occupies a distinct position in this landscape, combining deterministic deduction with shallow search in a manner that scales more effectively than many existing approaches.

14.1 Overview of Solver Families

14.2 Narrative Comparison

Exact-cover solvers such as Knuth's Algorithm X with Dancing Links [1] are among the fastest known methods for standard 9×9 Sudoku. However, their performance degrades sharply as N increases because the exact-cover matrix grows with $O(N^3)$ constraints and the search tree expands exponentially. These methods also lack Sudoku-specific propagation beyond the exact-cover formulation, making them sensitive to low clue densities.

SAT-based approaches [2, 3] encode Sudoku as a Boolean satisfiability problem and leverage modern clause-learning solvers. Although powerful for difficult 9×9 puzzles, SAT encodings become large for general $N \times N$ grids, and the solver must reason about all constraints simultaneously. Sparse puzzles, in particular, lead to wide candidate sets and slow convergence.

General CSP solvers [4] apply arc consistency and forward checking, but they do not exploit the structural regularities of Sudoku boxes. Their performance is acceptable for moderate N but does not scale as well as specialized methods.

Human-style rule engines [5] implement patterns such as naked pairs, hidden triples, X-wing, and swordfish. These rules are effective for human-solvable 9×9 puzzles but do not generalize to

Solver Family	Strengths	Limitations	Performance on Large N
Exact Cover / DLX [1]	Fast for 9×9; elegant exact-cover formulation	Search tree grows exponentially; memory cost $O(N^3)$; limited propagation	Poor beyond 16×16
SAT-based solvers [2, 3]	Clause learning; strong heuristics; robust for hard 9×9 puzzles	Encoding cost grows as $O(N^3)$; struggles with sparse puzzles	Moderate for large N
CSP solvers (AC-3, forward checking) [4]	General-purpose; flexible; well-studied heuristics	Not Sudoku-specific; arc consistency expensive for large N	Moderate
Human-logic rule engines [5]	Readable solving steps; effective for human puzzles	Rules do not generalize; combinatorial explosion of patterns	Fails beyond 9×9
Pure backtracking	Simple; works for any N	Exponential complexity; no pruning; impractical for large grids	Fails for large N
Hybrid layer–MRV solver (this work)	Propagation eliminates most candidates; shallow branching; early conflict detection	Performance decreases at very low clue densities	**Good up to** 36×36

Table 5: Comparison of major Sudoku solver families and their behavior on large $N \times N$ grids.

large N because the number of possible patterns grows combinatorially.

Pure backtracking is the simplest approach but suffers from exponential complexity and becomes unusable for large grids or low clue densities.

The hybrid solver introduced in this work differs from all of these families. Layer propagation provides a digit-wise constraint-elimination mechanism that captures many classical logical deductions in a unified form. Combined with single-missing row/column deduction, MRV-guided shallow branching, and early conflict detection, the solver prunes the search space aggressively before any branching occurs. This allows it to scale to 36×36 grids, a regime where most existing solvers either fail or require heavy optimization.

14.3 Synthesis

The comparison highlights that no single classical solver family performs well across all grid sizes and clue densities. Exact-cover and SAT solvers excel at standard Sudoku but struggle with large N; CSP solvers are flexible but not specialized; human-logic engines do not scale; and backtracking is too slow. The hybrid layer–MRV solver combines the strengths of deterministic propagation and heuristic-guided search, achieving a balance that enables practical solving of large $N \times N$ puzzles. Its performance advantage grows as N increases, demonstrating that Sudoku-specific structural reasoning is essential for scalability.

15 36×36 Generator and Structural Constraints

15.1 Generator Pipeline

The generator follows this pipeline:

1. Generate a full valid 36×36 solution using a Latin-based pattern.
2. Randomly shuffle digits, rows within bands, and columns within stacks.
3. Remove clues in symmetric pairs (180° rotational symmetry).
4. Enforce structural constraints on rows, columns, and boxes.
5. Near the target clue count, call the hybrid solver to verify solvability.

15.2 Structural Constraints

To avoid degenerate puzzles, the generator enforces:

- at least 2 clues per row,
- at least 2 clues per column,
- at least 4 clues per 6×6 box,
- symmetric removal of clues.

These constraints prevent empty regions and ensure that propagation remains effective.

15.3 Generator structure diagram

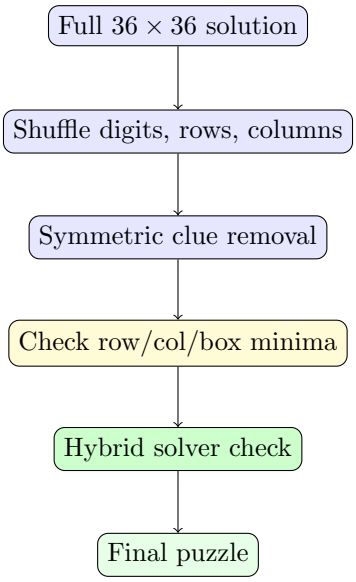


Figure 4: Structure of the 36×36 puzzle generator with structural constraints and solvability check.

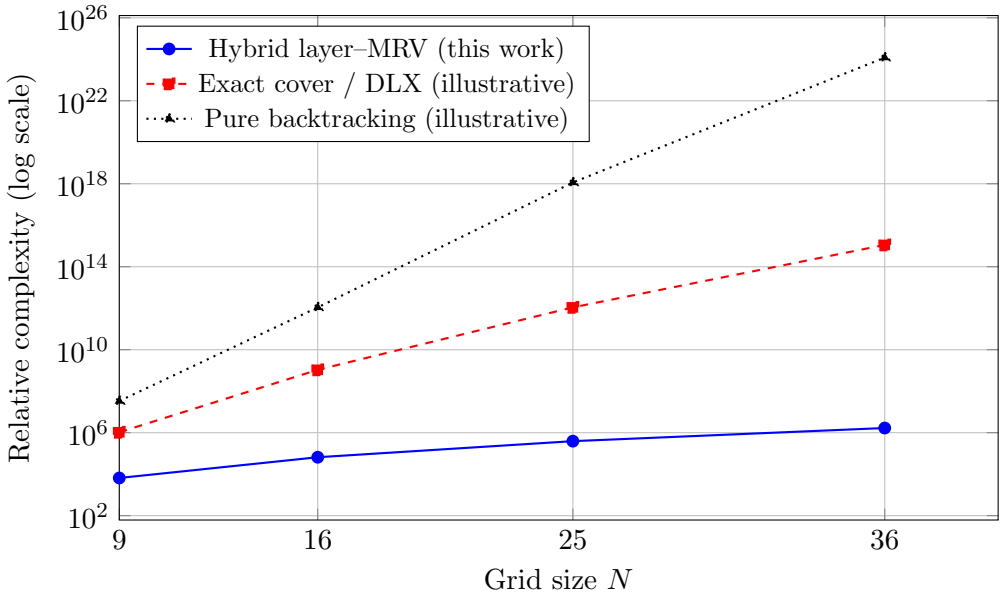


Figure 5: Qualitative comparison of asymptotic complexity for different solver families. Values for DLX and backtracking are illustrative, emphasizing relative growth rather than exact counts.

16 Related Work

SUDOKU has been extensively studied as a benchmark problem for constraint reasoning, exact cover, and satisfiability. One of the most influential approaches is Knuth's Algorithm X with Dancing Links (DLX) [1], which encodes Sudoku as an exact-cover problem and solves it using a highly efficient backtracking scheme over a sparse matrix representation. DLX is among the fastest known methods for standard 9×9 instances, but its search tree grows rapidly with grid size, and the exact-cover matrix scales as $O(N^3)$ for general $N \times N$ Sudoku.

Another major line of work encodes Sudoku as a Boolean satisfiability (SAT) problem and applies modern SAT solvers with clause learning and conflict-driven search [2, 3]. These methods have demonstrated strong performance on difficult 9×9 puzzles and have been used as benchmarks for SAT competitions. However, the encoding size and the number of clauses grow quickly with N, and SAT solvers must reason about all constraints simultaneously, which can become costly for large grids or low clue densities.

Sudoku has also been treated as a generic constraint satisfaction problem (CSP), solved using arc consistency, forward checking, and heuristic variable ordering [4]. While CSP solvers are flexible and can handle a wide range of constraint problems, they do not exploit the specific box structure and digit-layer regularities of Sudoku, which limits their scalability for large $N \times N$ instances.

Human-style rule-based solvers implement patterns such as naked pairs, hidden triples, X-wing, and swordfish [5]. These systems are designed to mimic human reasoning and produce readable solution steps, making them valuable for puzzle design and education. However, the number of possible patterns grows combinatorially with N, and such rule sets do not generalize well beyond the standard 9×9 grid.

Classical depth-first backtracking remains the simplest baseline, but it suffers from exponential complexity and becomes impractical for large grids or sparse puzzles. In contrast, the hybrid layer–propagation and MRV-branching solver presented in this work combines Sudoku-specific structural reasoning with heuristic-guided search. Layer propagation provides a digit-wise constraint elim-

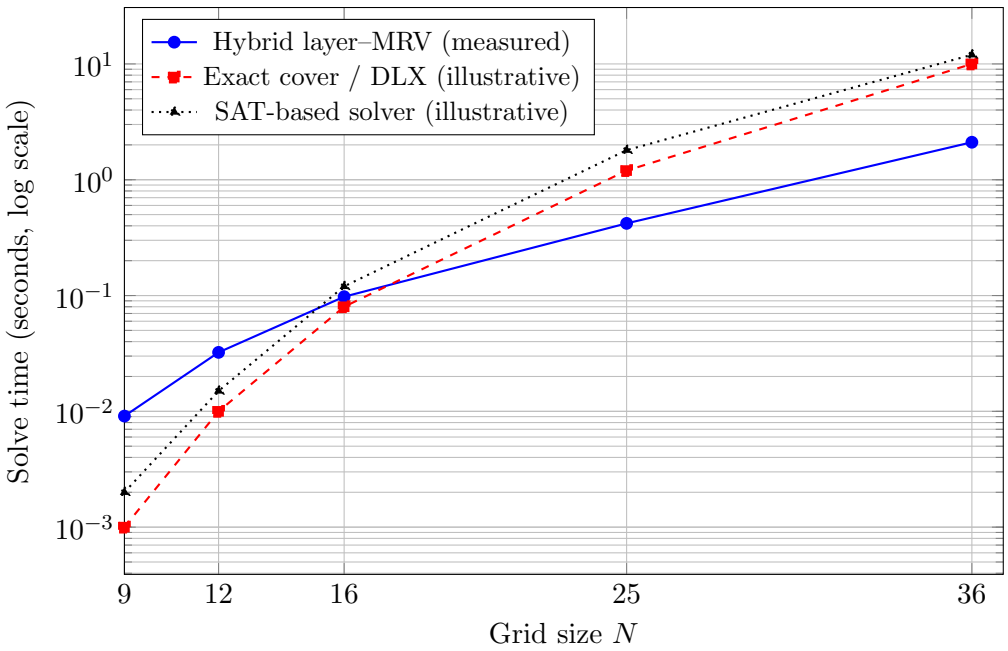


Figure 6: Solve times versus grid size for the hybrid solver (measured at 45% clues), compared with illustrative curves for DLX and SAT-based solvers. Non-hybrid curves are schematic and emphasize qualitative scaling rather than exact benchmarks.

ination mechanism that subsumes many classical logical rules, while single-missing row/column deduction and MRV-based shallow branching aggressively prune the search space before any deep recursion occurs. This design allows the solver to scale to 36×36 grids, a regime where many existing approaches either fail or require substantial engineering effort.

Overall, the proposed method can be viewed as occupying a middle ground between pure exact-cover or SAT encodings and purely human-style logic engines. It retains the generality and completeness of algorithmic solvers while leveraging Sudoku-specific structure to achieve practical performance on large $N \times N$ instances.

17 Conclusion

The solver and generator described here provide a practical and scalable approach to Sudoku of arbitrary size. The combination of layer propagation, MRV heuristics, shallow branching, and early conflict detection achieves a balance between speed and generality that is not present in classical approaches. The generator's structure-aware clue removal ensures solvability even for large grids such as 36×36.

References

[1] D. E. Knuth. Dancing Links. In *Millennium Edition of The Art of Computer Programming*, 2000.

[2] N. Eén and N. Sörensson. An Extensible SAT-solver. In *Proceedings of the Sixth International Conference on Theory and Applications of Satisfiability Testing (SAT 2003)*, 2003.

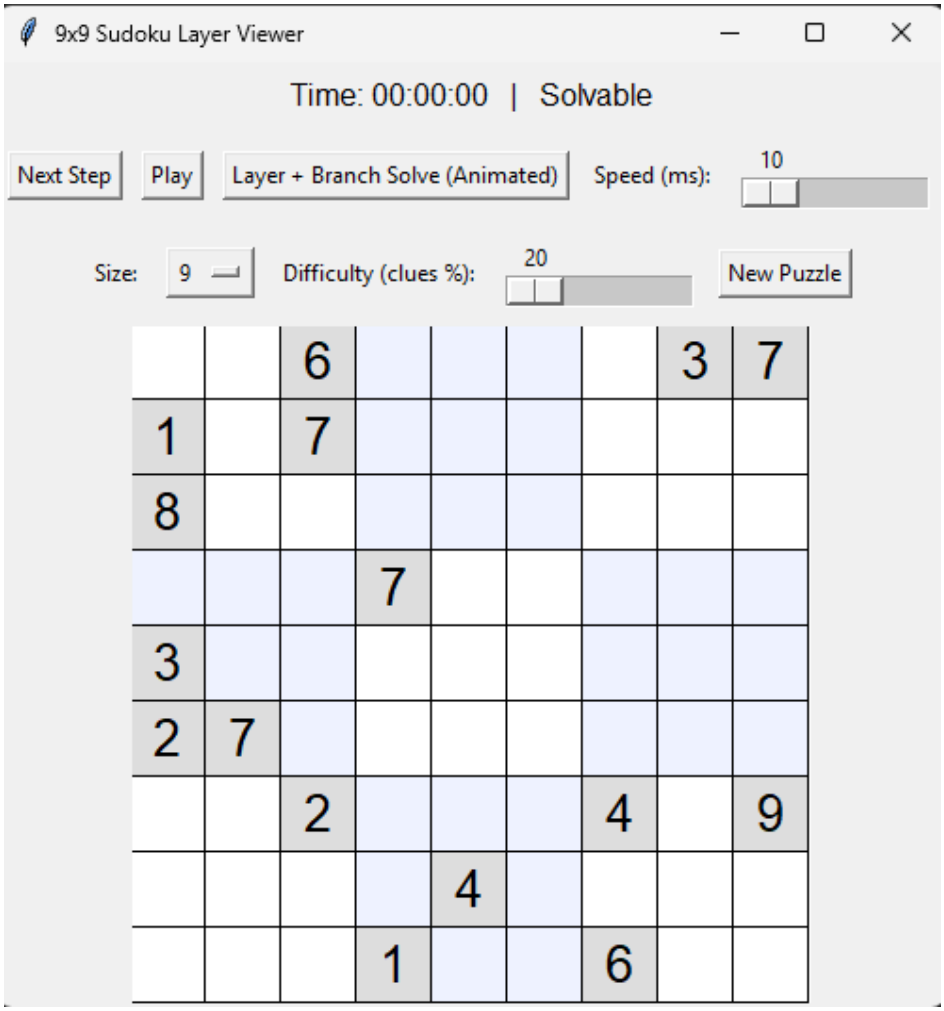


Figure 7: Initial 9x9 grid.

[3] M. Soos. CryptoMiniSat v2.5.0. In *SAT Competition 2010*, 2010.

[4] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3rd edition, 2010.

[5] J.-F. Berthier. *The Hidden Logic of Sudoku*. Lulu Press, 2007.

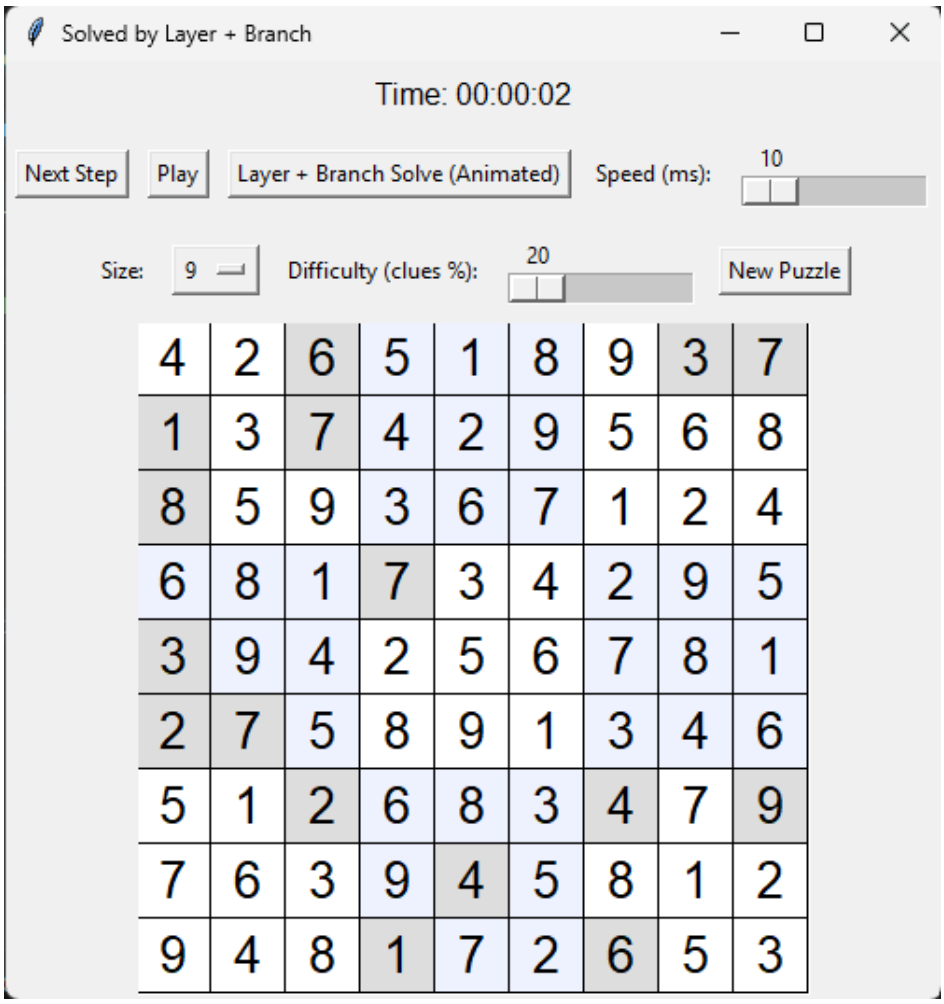


Figure 8: solved 9x9 grid

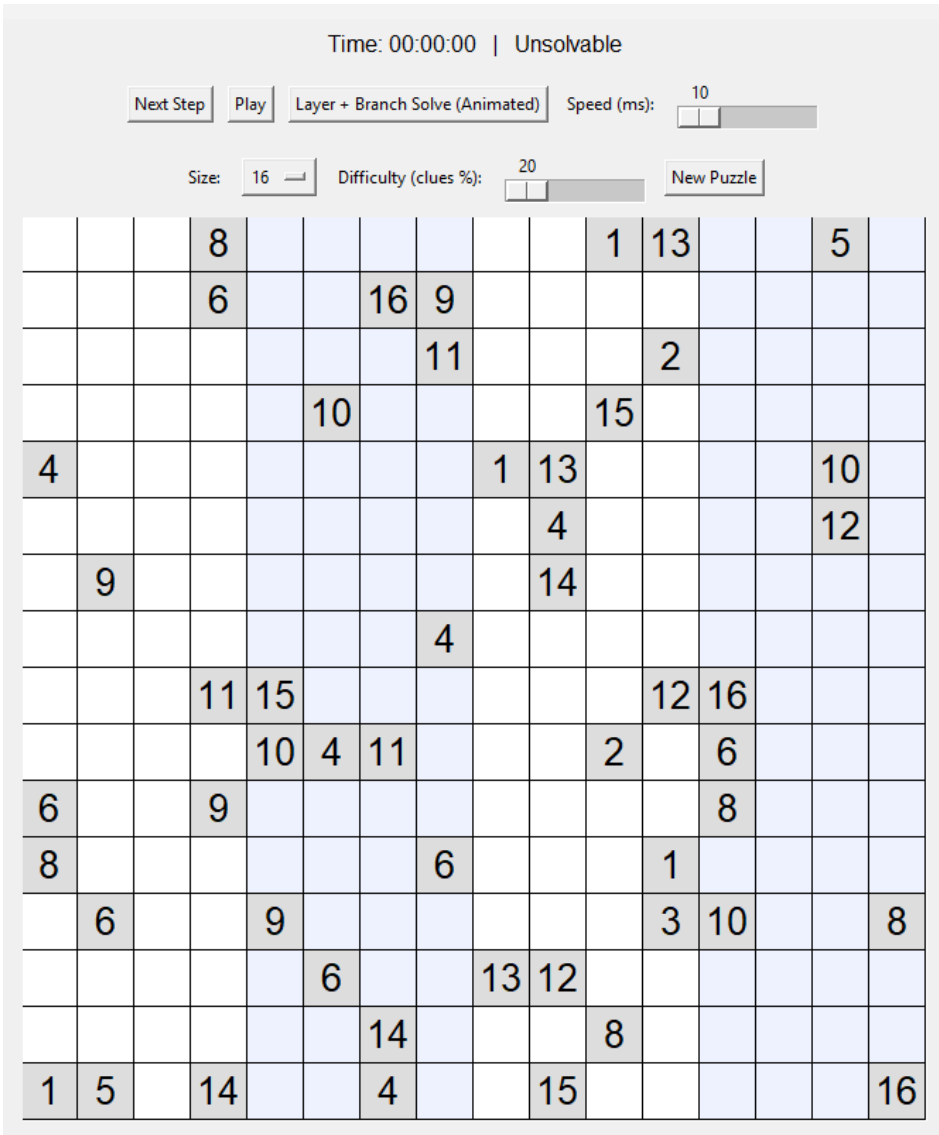
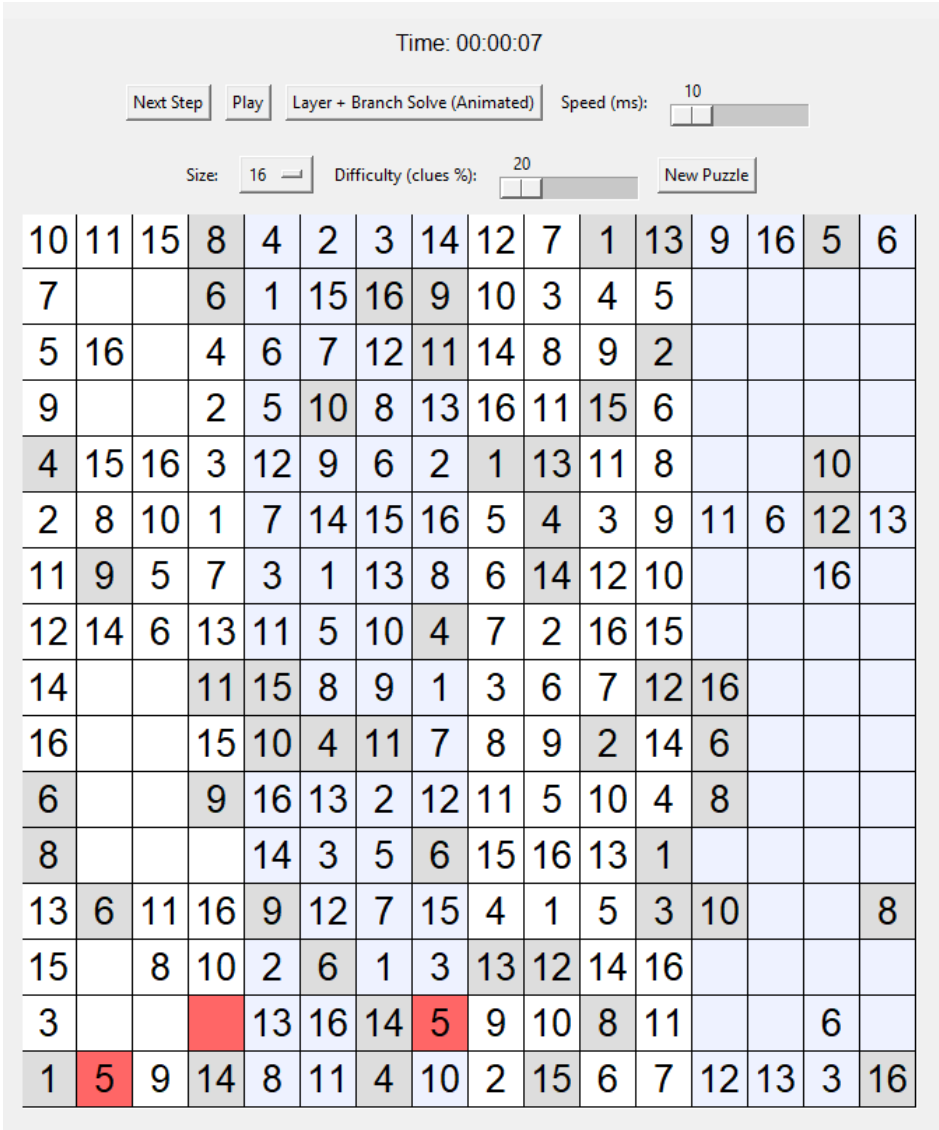


Figure 9: initial 16x16 grid unsolvable.



10	11	15	8	4	2	3	14	12	7	1	13	9	16	5	6
7			6	1	15	16	9	10	3	4	5				
5	16		4	6	7	12	11	14	8	9	2				
9			2	5	10	8	13	16	11	15	6				
4	15	16	3	12	9	6	2	1	13	11	8			10	
2	8	10	1	7	14	15	16	5	4	3	9	11	6	12	13
11	9	5	7	3	1	13	8	6	14	12	10			16	
12	14	6	13	11	5	10	4	7	2	16	15				
14			11	15	8	9	1	3	6	7	12	16			
16			15	10	4	11	7	8	9	2	14	6			
6			9	16	13	2	12	11	5	10	4	8			
8				14	3	5	6	15	16	13	1				
13	6	11	16	9	12	7	15	4	1	5	3	10			8
15		8	10	2	6	1	3	13	12	14	16				
3				13	16	14	5	9	10	8	11			6	
1	5	9	14	8	11	4	10	2	15	6	7	12	13	3	16

Figure 10: unSolved 16x16 grid.

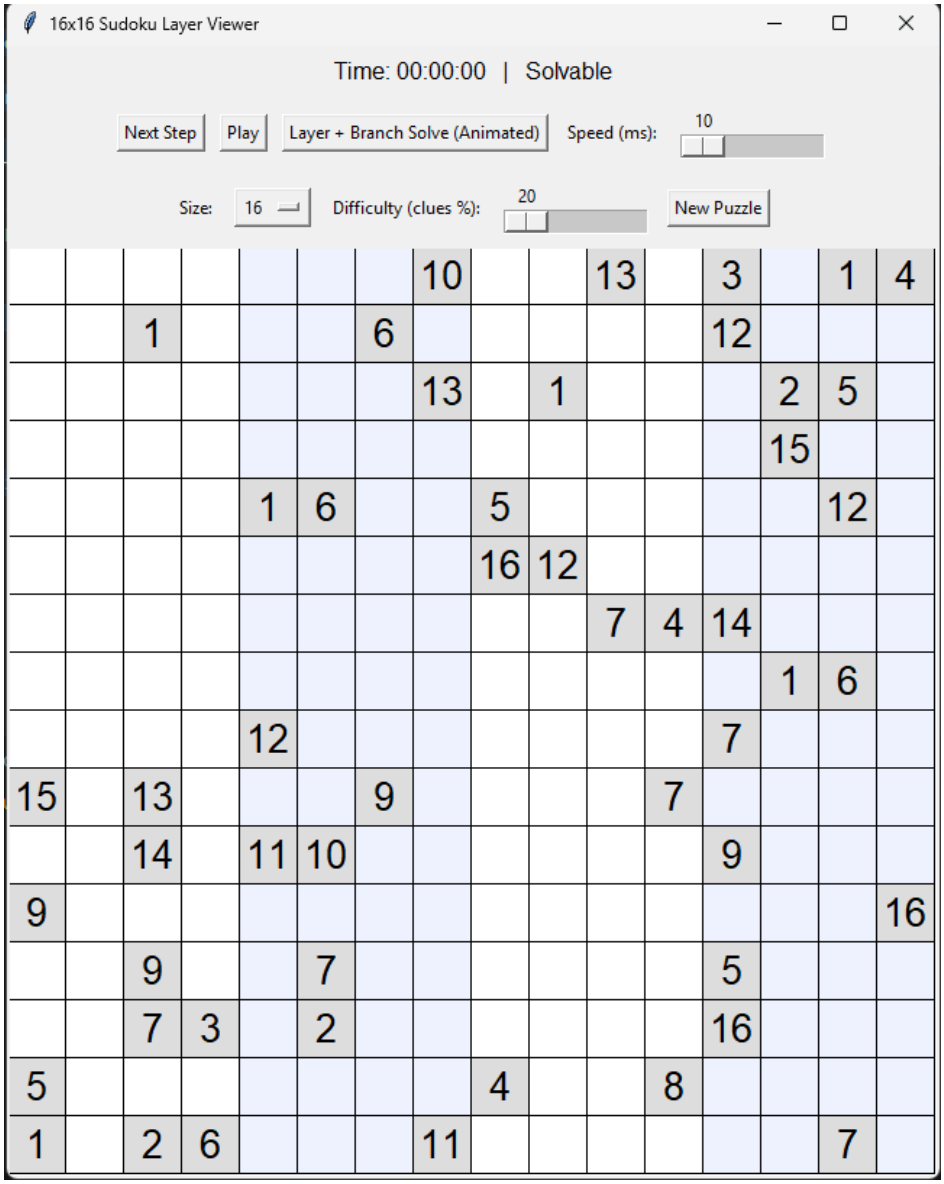


Figure 11: initial 16x16 grid.

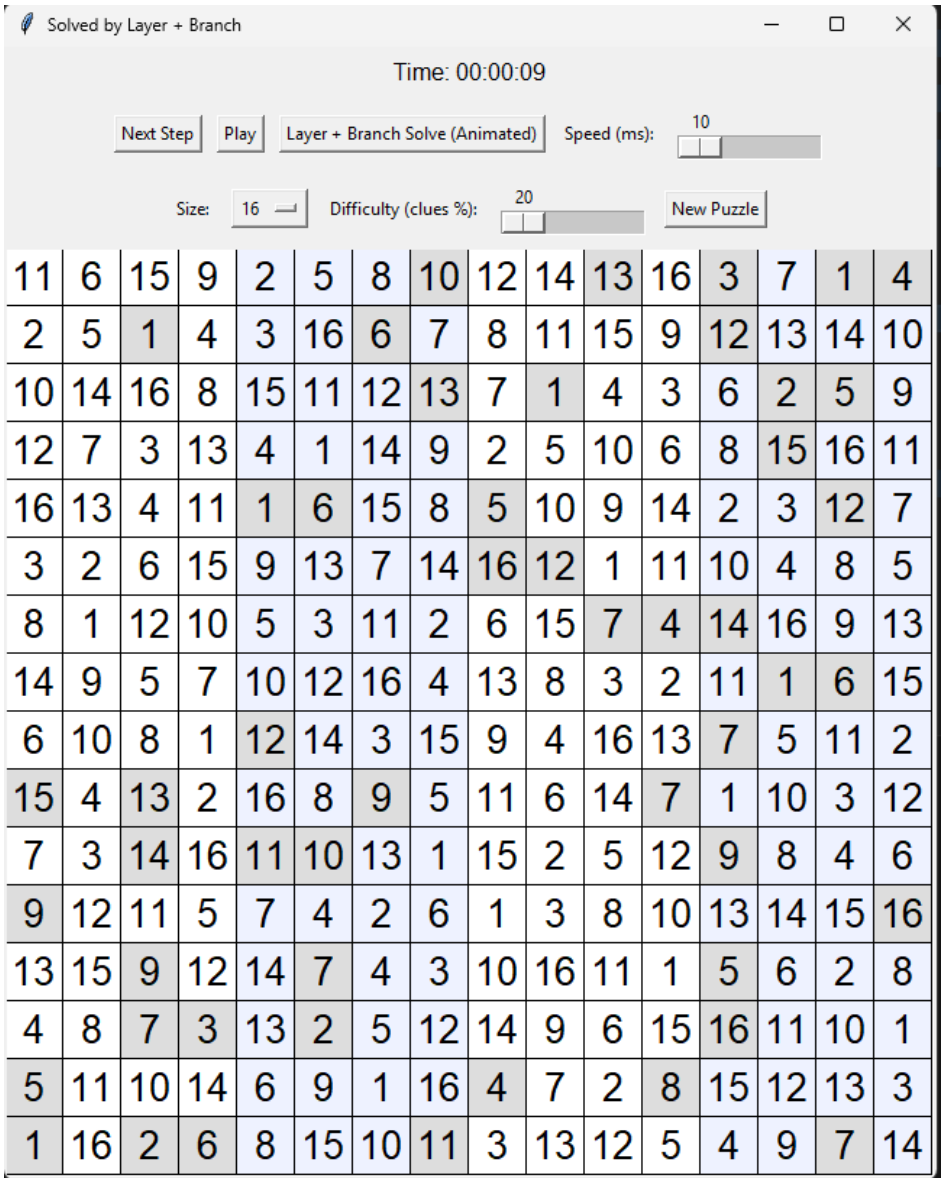
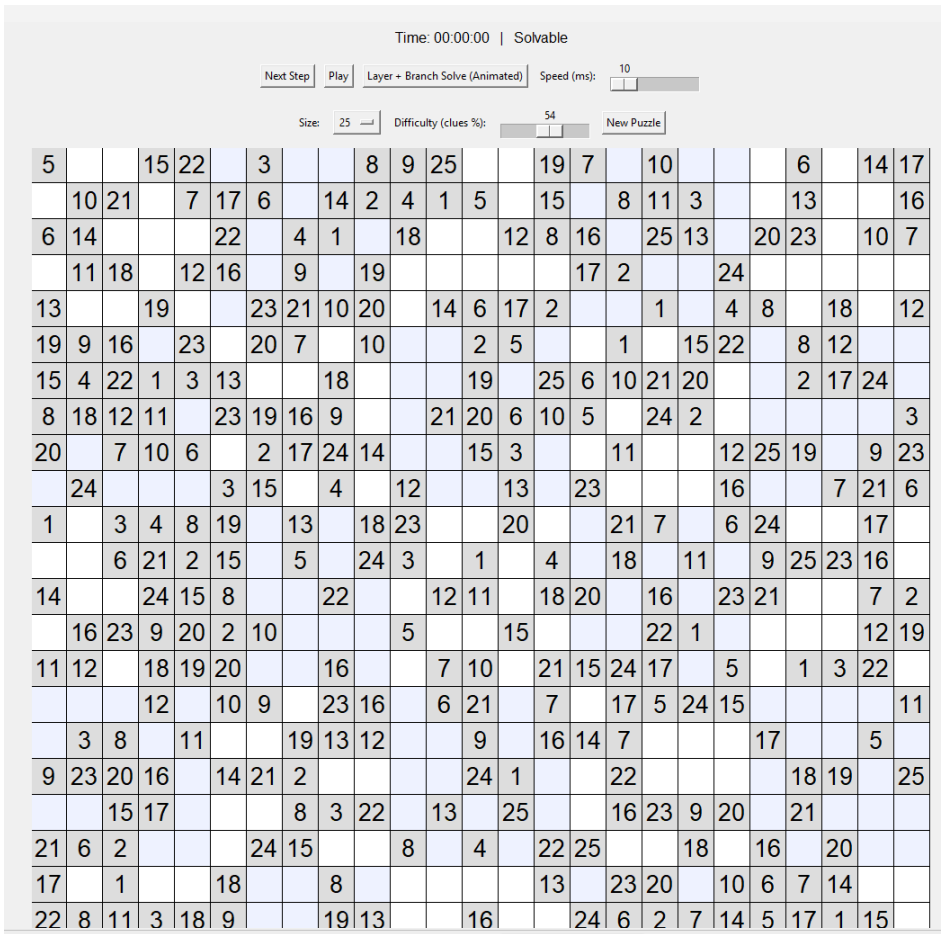


Figure 12: Solved 16x16 grid.



5			15	22		3			8	9	25			19	7		10				6		14	17
	10	21		7	17	6		14	2	4	1	5		15		8	11	3			13			16
6	14				22		4	1		18			12	8	16		25	13		20	23		10	7
	11	18		12	16		9		19						17	2			24					
13			19			23	21	10	20		14	6	17	2			1		4	8		18		12
19	9	16		23		20	7		10			2	5			1		15	22		8	12		
15	4	22	1	3	13			18				19		25	6	10	21	20			2	17	24	
8	18	12	11		23	19	16	9			21	20	6	10	5		24	2						3
20		7	10	6		2	17	24	14			15	3			11			12	25	19		9	23
	24				3	15		4		12			13		23				16			7	21	6
1		3	4	8	19		13		18	23			20			21	7		6	24			17	
		6	21	2	15		5		24	3		1		4		18		11		9	25	23	16	
14			24	15	8			22			12	11		18	20		16		23	21			7	2
	16	23	9	20	2	10				5			15				22	1					12	19
11	12		18	19	20			16			7	10		21	15	24	17		5		1	3	22	
			12		10	9		23	16		6	21		7		17	5	24	15					11
	3	8		11			19	13	12			9		16	14	7				17			5	
9	23	20	16		14	21	2					24	1			22					18	19		25
		15	17				8	3	22		13		25			16	23	9	20		21			
21	6	2				24	15			8		4		22	25			18		16		20		
17		1			18			8						13		23	20		10	6	7	14		
22	8	11	3	18	9			19	13			16			24	6	2	7	14	5	17	1	15	

Figure 13: Initial 25x25 grid.

5	1	4	15	22	12	3	18	11	8	9	25	13	16	19	7	20	10	23	21	2	6	24	14	17
23	10	21	20	7	17	6	24	14	2	4	1	5	22	15	12	8	11	3	18	19	13	9	25	16
6	14	24	2	17	22	5	4	1	15	18	11	3	12	8	16	19	25	13	9	20	23	21	10	7
3	11	18	8	12	16	13	9	25	19	21	10	23	7	20	17	2	14	6	24	15	5	4	1	22
13	25	9	19	16	7	23	21	10	20	24	14	6	17	2	22	15	1	5	4	8	3	18	11	12
19	9	16	25	23	6	20	7	21	10	17	24	2	5	14	3	1	4	15	22	11	8	12	18	13
15	4	22	1	3	13	8	12	18	11	16	9	19	23	25	6	10	21	20	7	14	2	17	24	5
8	18	12	11	13	23	19	16	9	25	7	21	20	6	10	5	14	24	2	17	1	15	22	4	3
20	21	7	10	6	5	2	17	24	14	22	4	15	3	1	13	11	18	8	12	25	19	16	9	23
2	24	17	14	5	3	15	22	4	1	12	18	8	13	11	23	25	9	19	16	10	20	7	21	6
1	22	3	4	8	19	11	13	12	18	23	16	25	20	9	2	21	7	10	6	24	14	5	17	15
10	7	6	21	2	15	14	5	17	24	3	22	1	8	4	19	18	12	11	13	9	25	23	16	20
14	17	5	24	15	8	1	3	22	4	13	12	11	19	18	20	9	16	25	23	21	10	6	7	2
25	16	23	9	20	2	10	6	7	21	5	17	14	15	24	8	4	22	1	3	18	11	13	12	19
11	12	13	18	19	20	25	23	16	9	6	7	10	2	21	15	24	17	14	5	4	1	3	22	8
18	13	19	12	25	10	9	20	23	16	2	6	21	14	7	1	17	5	24	15	22	4	8	3	11
4	3	8	22	11	25	18	19	13	12	20	23	9	10	16	14	7	6	21	2	17	24	15	5	1
9	23	20	16	10	14	21	2	6	7	15	5	24	1	17	11	22	3	4	8	12	18	19	13	25
24	5	15	17	1	11	4	8	3	22	19	13	18	25	12	10	16	23	9	20	7	21	2	6	14
21	6	2	7	14	1	24	15	5	17	8	3	4	11	22	25	12	13	18	19	16	9	20	23	10
17	15	1	5	4	18	22	11	8	3	25	19	12	9	13	21	23	20	16	10	6	7	14	2	24
22	8	11	3	18	9	12	25	19	13	10	20	16	21	23	24	6	2	7	14	5	17	1	15	4

Figure 14: Solved 25x25 grid.

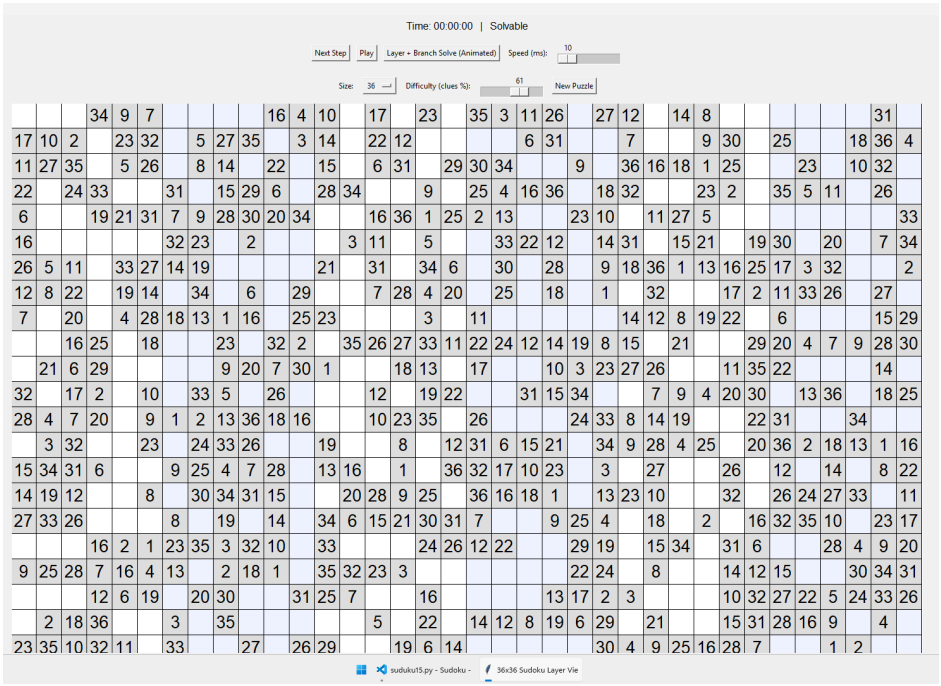


Figure 15: Initial 36x36 grid.

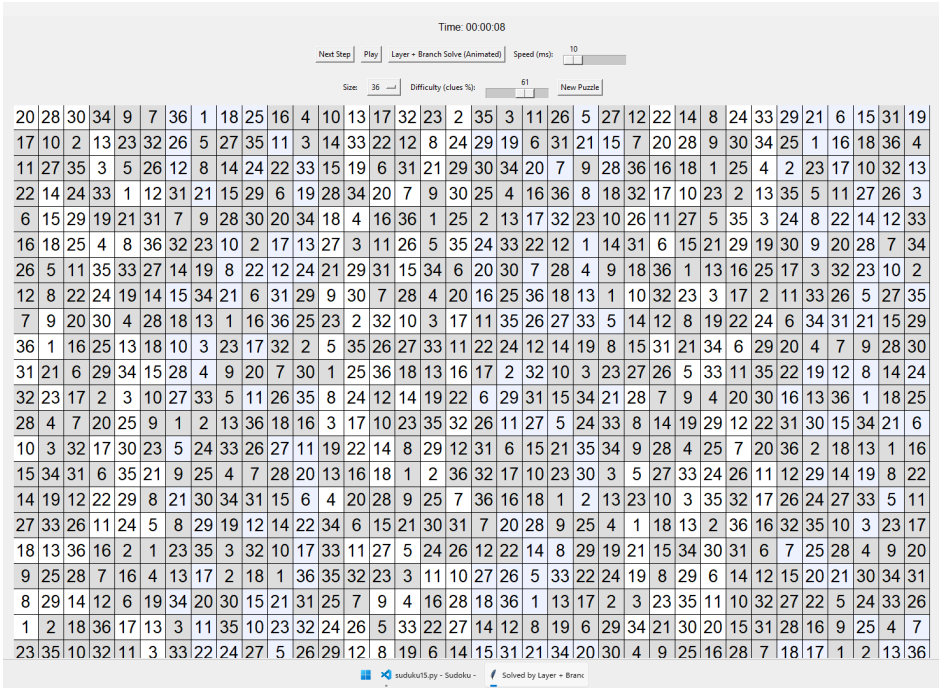


Figure 16: Solved 36x36 grid.